

The Mathlib contribution process

Ruth Plümer, June 9, 2026

1 What to contribute?

1.1 Where to start?

- Add lemmas to a theory that is already in Mathlib
- Extend a theory that is already in Mathlib
- Fix typos / style errors
- Work on an issue from the *issues list*
- Add a new theory, for example one of the *undergraduate topics missing in Mathlib*

1.2 Mathlib's values

- Trust & reliability
- Open source
- Maintainability
- General definitions
- Weak hypotheses - strong conclusions
- Classical logic instead of constructive logic
- As few issues for downstream projects as possible
- Accessibility for non-formalists
- Code of Conduct

1.3 A few words on style

Read the *style guide* and the *naming conventions*, in particular the *capitalization rules*!

Be sure to add a descriptive *documentation*! In particular:

- Your document should start with a descriptive docstring, containing
 - Copyright information
 - A list of imports
 - A descriptive summary of the contents and brief explanations of main definitions & main results
 - References
- Important definitions and key results should have a descriptive doc string
- Long proofs should contain comments to enhance readability

1.4 AI policy

Do not use AI for writing comments on Zulip and GitHub!

If you have used AI for code generation, indicate it clearly in the PR description & add the label **LLM-generated**!

You have to understand the code completely and be able to explain it yourself!

2 How to contribute?

Maintaining and version control of Mathlib are done using `git` on the platform GitHub.

If you have not used `git` before, check the *git guide for mathlib contributors*.

Typical workflow:

One-time setup:

1. Fork the Mathlib repository on GitHub.
2. Make a local clone of your fork and configure it properly (see instructions).

Daily workflow:

1. Create a new branch by typing `git switch -c my-new-branch` in the terminal
2. Make local changes
3. Commit your changes
4. Make sure everything works by running `lake build` and `lake exe mk_all`
5. Push your changes to GitHub
6. Go to GitHub
7. Click „Compare & pull request“
8. Fill in PR title & description
9. Click „Create pull request“
10. Wait for CI tests to finish & address potential issues

PR commit conventions: Title: `<type>(<optional-scope>): <subject>`

Description:

```
<body>
<NEWLINE>
<footer>
<NEWLINE>
<dependencies>
```

3 Behind the scenes

There are three „types of people“ who look at your PR: *anyone*, *reviewers* and *maintainers*.

Anyone can look at your pull request, suggest changes & leave an approving review if they feel your PR is ready to merge. **You can do that, too!**

If you want to review someone’s PR, make sure to read the *review guidelines* beforehand. In particular, pay attention to whether the style is good, the names of important theorems and definitions are correct, the documentation is sufficient and whether the result is in its most general form. If there are new definitions introduced, pay attention to whether the definitions actually do what they claim to do. Moreover, check whether generalizations of the theorems already exist in another place in Mathlib.

If you have made your comments, make sure to add the label `awaiting-author`.

Reviewers are experienced contributors with a history of helpful reviews. As of now, there are around 60 reviewers in total.

They do exactly the same as regular contributors, except that they can add the label `maintainer-merge` instead of simply approving your PR, which means that your PR will get checked by a maintainer earlier.

28 of the reviewers are also *maintainers*. They are the only people who can merge PRs. They will review the PRs once they have been approved or given the label `maintainer-merge`.

In most cases, they will directly approve the merging of your PRs, but sometimes, they might also *delegate* the merging to you, which usually means that you should address a few final comments and then type `bors merge` yourself.

In each case, the final merging (which takes about an hour) will be done automatically by a bot called `bors`.

3.1 The queueboard

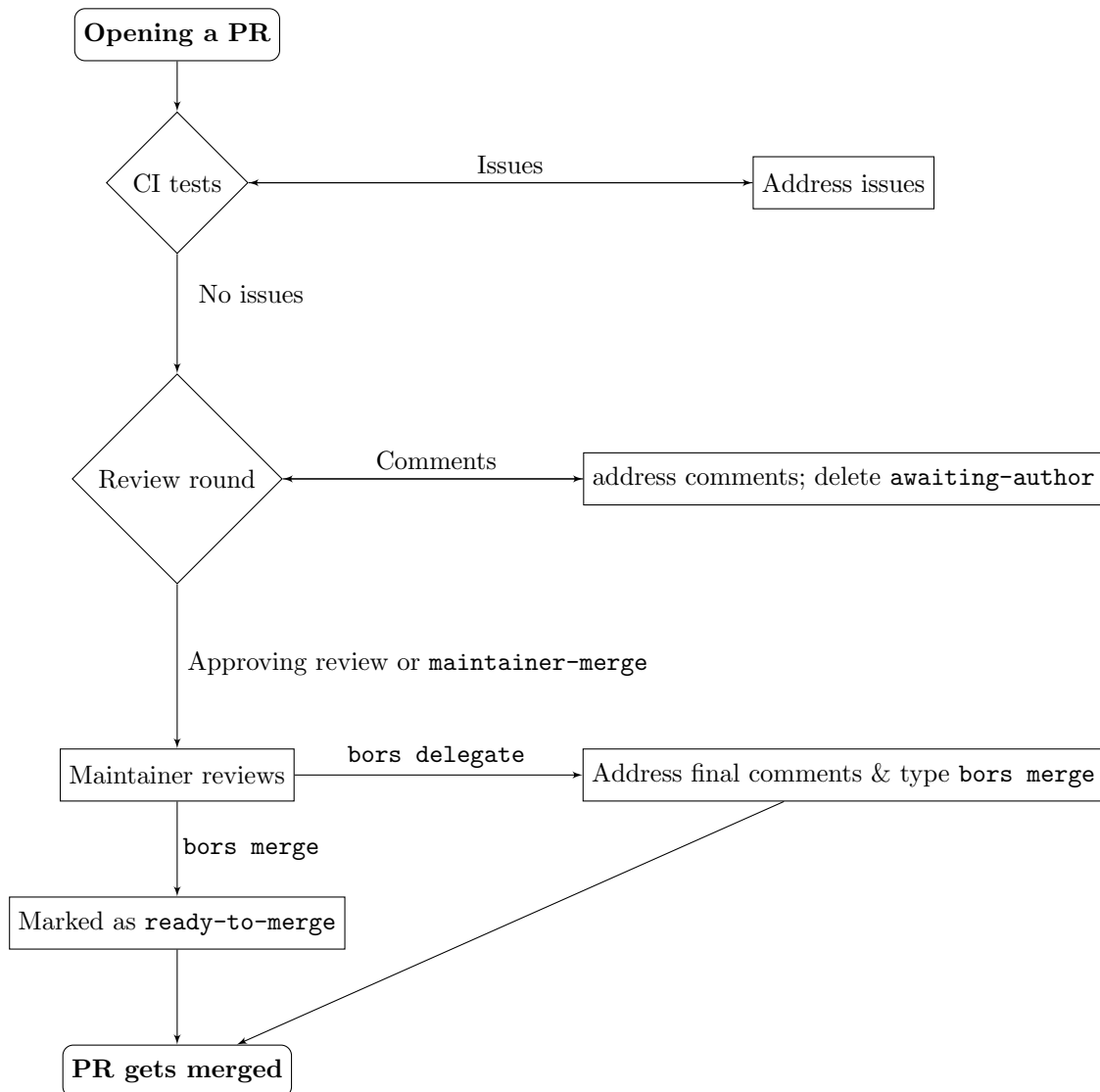
The queueboard (<https://leanprover-community.github.io/queueboard/>) is a useful tool for reviewers which helps organizing the huge number of open pull requests (As of now, there are 2587 open pull requests, 448 are awaiting review).

All PRs that are currently awaiting review (i.e. that have passed CI and do not have one of the labels `blocked-by-other-PR`, `merge-conflict`, `awaiting-author`, `RFC`, `awaiting-zulip`, `help-wanted`, **WIP** or `please-adopt`) are organized in the *review queue*.

A bot automatically assigns a reviewer to every new PR, respecting workload and area preferences (Mathlib is chronically in need of reviewers, so this cannot always be ensured).

You can check on the website of the queueboard if your PR is on the review queue and if not, why. Moreover, you can filter PRs for specific labels, authors, or files modified.

Lifecycle of a PR



4 Useful resources

- Mathematical interests of the Mathlib maintainers: <https://github.com/leanprover-community/mathlib4?tab=readme-ov-file#maintainers>
- CSLib, Lean’s computer science library: <https://github.com/leanprover/cslib>
- Physlib, Lean’s physics library: <https://github.com/leanprover-community/physlib/tree/master/Physlib>
- List of standalone projects depending on Mathlib: <https://reservoir.lean-lang.org/@leanprover-community/mathlib/dependents>
- Combinatorial game theory repository: <https://github.com/vihdzp/combinatorial-games>
- List of Mathlib issues: <https://github.com/leanprover-community/mathlib4/issues>
- List of undergraduate topics missing in Mathlib: https://leanprover-community.github.io/undergrad_todo.html
- Mathlib’s Code of Conduct: https://github.com/leanprover-community/mathlib4/blob/master/CODE_OF_CONDUCT.md
- Discussion on the Mathlib AI policy: <https://leanprover.zulipchat.com/#narrow/channel/113488-general/topic/PSA.3A.20mathlib.20AI.20policy/>
- Mathlib style guide: <https://leanprover-community.github.io/contribute/style.html>
- Mathlib naming conventions: <https://leanprover-community.github.io/contribute/naming.html>
- Documentation style guide: <https://leanprover-community.github.io/contribute/doc.html>
- Git Guide for Mathlib4 Contributors: <https://leanprover-community.github.io/contribute/git.html>
- Commit conventions: <https://leanprover-community.github.io/contribute/commit.html>
- Dealing with merge conflicts: <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/addressing-merge-conflicts/resolving-a-merge-conflict-on-github>
- List of Mathlib maintainers: <https://leanprover-community.github.io/teams/maintainers.html>
- List of Mathlib reviewers: <https://leanprover-community.github.io/teams/reviewers.html>
- Review guidelines: <https://leanprover-community.github.io/contribute/pr-review.html>
- Queueboard: <https://leanprover-community.github.io/queueboard/>

Literatur

- [Baa+26] Anne Baanen u. a. „Growing Mathlib: maintenance of a large scale mathematical library“. In: *Intelligent Computer Mathematics*. Hrsg. von Valeria de Paiva und Peter Koepke. Cham: Springer Nature Switzerland, 2026, S. 51–70. ISBN: 978-3-032-07021-0.
- [Leaa] Lean Prover Community. *Contributing to Mathlib*. URL: <https://leanprover-community.github.io/contribute/index.html>.
- [Leab] Lean Prover Community. *How to contribute to Mathlib*. URL: <https://leanprover-community.github.io/contribute/how-to-contribute.html>.
- [Leac] Lean Prover Community. *Mathlib’s values*. URL: <https://leanprover-community.github.io/contribute/values.html>.
- [Rot26a] Michael B. Rothgang. *Behind the scenes: orchestrating Mathlib’s review load*. März 2026. URL: https://www.math.uni-bonn.de/people/rothgang/slides_bootcamp_orchestrating_reviews.pdf.
- [Rot26b] Michael B. Rothgang. *Tooling for reviewers and review etiquette*. Apr. 2026. URL: https://www.math.uni-bonn.de/people/rothgang/slides_bootcamp_tooling_etiquette.pdf.